

2.1.1~2.1.5代码总结

一、数据加载与概览

1、加载CSV文件中的数据

`read_csv()`

`data = pd.read_csv('csv文件的路径')`

如果文件中的列名是中文，则要加上：`encoding='gbk'` 参数

```
# 加载数据集并显示数据集的前五行 1分
data = pd.read_csv('auto-mpg.csv')

# 加载数据集并指定编码为gbk
data = pd.read_csv('medical_data.csv', encoding='gbk')
```

2、加载excel文件中的数据

`read_excel()`

`data = pd.read_excel('excel文件的路径')`

```
#读取一个Excel文件，并将读取到的数据存储在变量data中
data = pd.read_excel('大学生低碳生活行为的影响因素数据集.xlsx')
```

3、显示前5行数据

`.head()`

```
print(data.head())
```

4、显示数据结构和基本信息

`info()`

```
print(data.info())
```

5、统计数据表行数

`shape[0]`

`data.shape[0]`

6、查看数据类型

.dtypes

```
print(data.dtypes)
```

二、数据预处理

1、数据类型转换

方法1:

.astype(新的数据类型) # int: 整数; float: 小数; string: 字符

data['列名'] = data['列名'].astype(新的数据类型)

方法2:

data['列名'] = pd.to_numeric(data['列名'], errors='coerce')

```
# 将 'horsepower' 列转换为数值类型，并（删除）处理转换中的异常值 1分
data['horsepower'] = pd.to_numeric(data['horsepower'], errors='coerce')
data = data.dropna(subset=['horsepower'])
```

2、修改列名

data.rename(columns={'old列名': 'new列名'}, inplace=True)

```
# 修改列名
data.rename(columns={'病人ID': '患者ID'}, inplace=True)
```

三、数据清洗

1、检查缺失值

.isnull() # 判断是不是缺失值，返回结果是：true; 不是：false.

.isnull().sum() # sum() 计算每一列缺失值的数量

```
print(data.isnull().sum())
```

2、删除缺失值所在行

.dropna()

```
data = data.dropna()
```

3、检查重复值或重复行

`.duplicated()` # 判断是不是重复的行，返回结果是：true; 不是：false.

`.duplicated().sum()` # `sum()` 计算总的重复行的数量

```
print(data.duplicated().sum())
```

4、删除重复行

`.drop_duplicates()`

```
data = data.drop_duplicates()
```

5、清除异常值

`.quantile()` # 计算数据分位数

IQR方法：

1. 先计算出Q1: `Q1 = data[numeric_cols].quantile(0.25)`
2. 再计算出Q3: `Q3 = data[numeric_cols].quantile(0.75)`
3. 最后计算出: `IQR = Q3 - Q1`
4. 异常值范围: `(data[numeric_cols] < (Q1 - 1.5 * IQR)) | (data[numeric_cols] > (Q3 + 1.5 * IQR))`

使用IQR处理异常值

```
Q1 = data[numeric_cols].quantile(0.25)
```

```
Q3 = data[numeric_cols].quantile(0.75)
```

```
IQR = Q3 - Q1
```

移除异常值

```
data_cleaned = data[~((data[numeric_cols] < (Q1 - 1.5 * IQR)) | (data[numeric_cols] > (Q3 + 1.5 * IQR))).any(axis=1)]
```

条件筛选法：

`data[(逻辑条件)]`

```
data = data[(data['诊断延迟'] >= 0) & (data['年龄'] > 0) & (data['年龄'] < 120)]
```

删除不合理的数据

```
data = data[(data['诊断延迟'] >= 0) & (data['年龄'] > 0) & (data['年龄'] < 120)]
```

四、数据标准化/归一化

1. 标准化：

```
.fit_transform(data['列名'])
```

```
scaler = StandardScaler()
```

```
data[numerical_features] = scaler.fit_transform(data[numerical_features])
```

2. 归一化:

```
.fit_transform(data['列名'])
```

```
scaler = MinMaxScaler()
```

```
data_cleaned[numeric_cols] = scaler.fit_transform(data_cleaned[numeric_cols])
```

五、特征工程（数据标注）

方法1：先写好列名，再从数据表中提取数据

```
selected_features = ['cylinders', 'displacement', 'horsepower', 'weight', 'acceleration', 'model  
year', 'origin']  
x = data[selected_features]  
y = data['mpg']
```

方法2：删除目标列，剩下的做为特征

```
# 设定目标变量  
target_variable = 'seriousDlqin2yrs'  
  
# 定义特征和目标  
x = data_cleaned.drop(columns=['Unnamed: 0', target_variable]) #1分  
y = data_cleaned[target_variable]
```

六、数据集划分

方法1：对 X, y 进行划分

```
from sklearn.model_selection import train_test_split  
  
# 划分数据（训练集占80%）  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

方法2：对整个数据集划分

```
# 填充缺失值  
data_filled = data.apply(lambda x: x.fillna(x.mode()[0]))  
  
# 划分数据（测试集占比20%）  
train_data, test_data = train_test_split(data_filled, test_size=0.2, random_state=42)
```

七、数据集合并输出

```
pd.concat([X, y])
```

```
# 合并处理后得数据，并将其保存（保存中不用额外创建索引）
cleaned_data = pd.concat([X, y], axis=1)
cleaned_data.to_csv('2.1.2_cleaned_data.csv', index=False)
```

八、数据可视化（图表）

柱状图：

```
# 统计治疗结果分布
treatment_outcome_distribution = data.groupby('疾病类型')['治疗结果'].value_counts().unstack()
# 绘制柱状图
treatment_outcome_distribution.plot(kind='bar', stacked=True)
```

散点图：

```
# 绘制散点图
plt.scatter(data['年龄'], data['疾病严重程度'])
```

饼图：

```
# 统计不同健身频率的分布情况
exercise_frequency_counts = data_cleaned['How often do you exercise?'].value_counts()

# 绘制饼图
plt.figure(figsize=(10, 6))
exercise_frequency_counts.plot.pie(autopct='%1.1f%%', startangle=90,
colors=plt.cm.Paired.colors)
```